

Porting MOM6 to GPUs with standard parallelism

One source code for CPUs and GPUs, and a few directives

Jorge Luis Gálvez Vallejo (NCI)

with Edward Yang (ACCESS-NRI), Utheri Wagura (SAIC) & Marshall
Ward (NOAA-GFDL)

jorge.galvezvallejo@anu.edu.au

NCI • ACCESS-NRI

Australian National University



ROADMAP

Agenda

- 1 MOM6 and the GPU problem
- 2 Standard parallelism
- 3 The dynamical core
- 4 CPU & GPU parity
- 5 Results

TOC

1

MOM6 and the GPU problem

1 MOM6: a community ocean model

- **200k lines** of modern Fortran
- MPI-parallelized, little threading
- Used in climate models and by many partners worldwide
- Core component of **Australia's** climate models

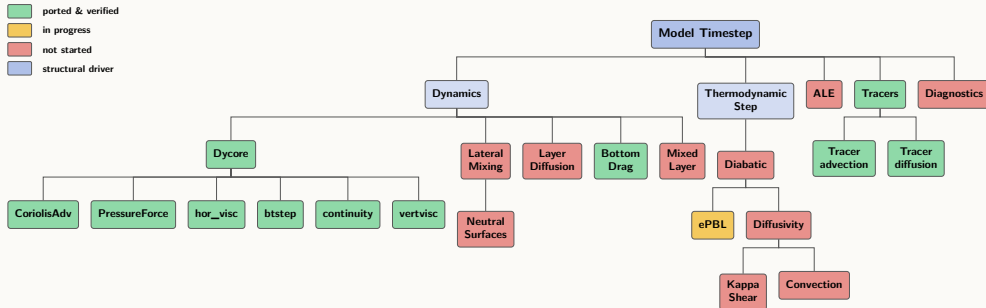
Fortran MPI climate

Two layers

- **FMS** — MPI comms, I/O, scaling
- **MOM6** — dynamical core, parametrizations, physics

*This GPU port is an NCI effort **in support of** NOAA-GFDL (MOM6's home) and ACCESS-NRI — a collaboration, not a solo rewrite.*

Where the work lives, and what is ported



Dynamical core: ported & verified. Parametrizations / physics *underway* — tracers done; diabatic, ALE and lateral mixing in progress.

1 Porting options

- **Rewrite in C/C++** — 200k lines, used worldwide; a rewrite is very expensive
 - Kokkos for GPU kernels (e.g. NCAR / E3SM efforts) — its own complex beast
- **OpenMP target** directives — standardized, broad compiler support
- **OpenACC** — very mature on NVIDIA; elsewhere partial (GCC) or Cray-only
- **Standard parallelism** (do concurrent) — plain Fortran, let the compiler do it

Our choice

Standard-language parallelism — elegant, portable, and it stays *Fortran* for the community.

1 Goals and conditions

- Code must run on GPUs
- **One** source for CPUs and GPUs
- GPU work must not degrade the CPU path
- Portability across vendors
- **Bitwise reproducibility** preserved

The contract

Identical bits across decompositions and devices — a hard requirement inherited from MOM6.

2

Standard parallelism



2 do concurrent: parallelism in the language

Standard Fortran

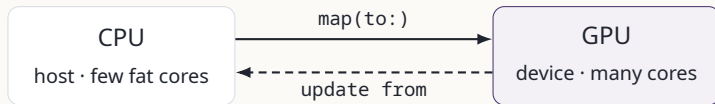
```
do concurrent (k=1:nk,j=1:nj,i=1:
  ni)
  c(i,j,k) = a(i,j,k)
end do
```

Directive equivalents

```
!$omp target teams loop collapse
  (3)
!$acc parallel loop collapse(3)
do k = 1, nk
  do j = 1, nj
    do i = 1, ni
      c(i,j,k) = a(i,j,k)
    end do
  end do
end do
```

- A **contract**: every iteration is independent and thread-safe; for offload, called procedures must be pure.
- Idiomatic, no vendor pragmas — and a routine containing `do concurrent` can itself stay pure.

2 Data movement: compute in the language, move with directives



interconnect (PCIe / NVLink)

- Compute stays in `do concurrent`; OpenMP target ... `map` keeps arrays **resident** on the device.
- Minimize transfers: work backwards from the kernels, keep data on the GPU between steps.
- OpenMP mapping is portable across NVIDIA, AMD and Intel.

2

do concurrent support across compilers

Each vendor offloads to *its* GPU — NVIDIA is the most mature, hands down

Compiler	DC → CPU	DC → GPU	Notes / flags
nvfortran (NVHPC)	Yes	Yes	most mature; -stdpar=gpu; managed mem; F2023 reduce/locality
ifx (Intel)	Yes	Yes	DC lowered to OpenMP; -fopenmp-target-do-concurrent; ifort EOL
ftn (Cray CCE)	Yes	Yes	AMD + NVIDIA, unified mem; -h thread_do_concurrent
flang/aof (AOCC)	Yes	No	x86/Zen only; -mp; → llvm-flang in 6.0
xlf (IBM XL)	Yes	Part.	POWER+NVLink only; <i>legacy</i> (Open XL)
frt (Fujitsu)	Yes	No	SVE auto-vec; -Kparallel; not DC-aware
gfortran (GCC)	Part.	No	generic autopar -ftree-parallelize-loops=N; not DC-aware
LLVM flang	Part.	Part.	emerging; -fdo-concurrent-to-openmp=host device
lfortran	Part.	Part.	beta; unified mem; not production-ready
nagfor (NAG)	No	No	serial; correctness oracle, not a perf path

2 Pain points and niceties

Pain

- Calling procedures across translation unit is painful
- The compiler picks the parallelism (blocks/threads) — can be suboptimal
- Performance is compiler- and architecture-dependent

Nice

- `pure` propagates up the call tree — compiler-checked safety
- Same construct drives CPU threads *and* GPU offload
- No vendor pragmas in the science code

3

The dynamical core



3

Split RK2 timestepping

`step_MOM_dyn_split_RK2` — repeated `n_timesteps` times

Predictor (build accel. at n)

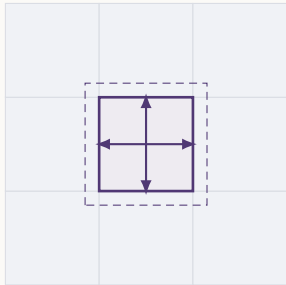
1. PressureForce $PF = -\partial_x M$
2. Coriolis + advection CA
3. $u_{bc} = CA + PF + \text{diffu}$
4. vert. visc. coeffs halo η
5. barotropic coupling
6. **btstep** — subcycle $\times n_{bt}$
7. $u^* = u^n + dt(u_{bc} + u_{bt})$
8. vertvisc (implicit, tridiag.)
9. continuity $\rightarrow h^*, uh, vh$
10. PressureForce(h^*)

Corrector (take the step)

11. horizontal_viscosity (Laplacian)
12. Coriolis + advection (u_{av})
13. $u_{bc} = CA + PF + \text{diffu}$
14. **btstep** — subcycle $\times n_{bt}$
15. $u^{n+1} = u^n + dt(u_{bc} + u_{bt})$
16. vertvisc (implicit, tridiag.)
17. continuity $\rightarrow h^{n+1}$ halo u, h
18. Coriolis post (diagnostics)

Each block is one or a few do concurrent kernels; halo passes (`pass_*`) interleave and overlap.

3 Domain decomposition & halo updates



nz not decomposed

- Horizontal $n_i \times n_j$ tiled across MPI ranks (FMS); the **vertical n_z is never split** — columns stay whole.
- Each tile carries a **halo** (width = stencil radius, 1–4 cells).
- `pass_var` / `pass_vector` refresh faces before each stencil.
- **GPU**: halos exchanged with `omp_offload` (GPUDirect) — no host staging.

$$\text{compute} \propto \text{volume } N/P \quad \text{comm} \propto \text{surface} \propto \sqrt{1/P}$$

3 Six algorithmic motifs

#	Motif	Work	Depth	GPU fit
1	Pointwise / map ($n_i n_j n_z$)	$\Theta(N)$	$\Theta(1)$	ideal
2	Horizontal / Laplacian stencil	$\Theta(N)$	$\Theta(1)$	great + halo
3	Tridiagonal column solve (Thomas)	$\Theta(N)$	$\Theta(n_z)$	good (jik)
4	Iterative column solve	$\Theta(N \cdot it)$	$\Theta(n_z \cdot it)$	harder (divergence)
5	Barotropic subcycle (2-D split-explicit)	$\Theta(H n_{bt})$	$\Theta(n_{bt})$	latency-bound
6	Halo exchange / comm	$\Theta(\text{surf})$	$\Theta(\log P)$	GPUDirect

$N = n_i n_j n_z$ cells $H = n_i n_j$ one level it = iteration count P = ranks

3 Motif 1 — Pointwise / map



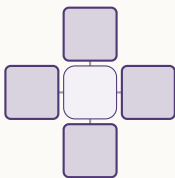
reads only (i, j, k)

embarrassingly parallel

```
do concurrent (k=1:nz, j=1:nj, i=1:ni)  
  C(i,j,k) = f( A(i,j,k), B(i,j,k) )  
end do
```

- EoS, scaling, tracer sources. Work $\Theta(N)$ · depth $\Theta(1)$ · no comm
- Memory-bound, perfect occupancy — the easy 80%.

3 Motif 2 — Horizontal / Laplacian stencil



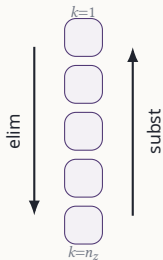
5-point: $i \pm 1, j \pm 1$

neighbour reads

```
do concurrent (k=1:nz, j, i)
  L = A(i-1,j,k) + A(i+1,j,k)      &
    + A(i,j-1,k) + A(i,j+1,k)      &
    - 4*A(i,j,k)
end do
```

- Horizontal viscosity, Coriolis+KE, ∂_x pressure. Work $\Theta(N)$ · depth $\Theta(1)$ · needs halo
- Memory-bound with neighbour reuse — coalescing matters.

3 Motif 3 — Tridiagonal column solve (Thomas)



- Implicit vertical viscosity / diffusion:
$$a_k x_{k-1} + b_k x_k + c_k x_{k+1} = d_k.$$
- Thomas: $O(n_z)$ **sequential** sweep per column; columns independent.
- Map $n_i n_j$ threads, each walking its own column $\Rightarrow$ the **jik** ordering.

Work $\Theta(N)$ · depth $\Theta(n_z)$ · columns fully parallel

3 Motif 4 — Iterative column solve

data-dependent iteration count

```
do concurrent (j, i)                ! columns parallel
  iter = 0
  do while (.not. converged .and. iter < max)
    ! update kappa(k) from shear; solve tridiagonal column; check
    residual
    iter = iter + 1
  end do
end do
```

- Shear-driven mixing (kappa-shear), implicit / convective adjustment.
- Work $\Theta(N \cdot it) \cdot \text{depth } \Theta(n_z \cdot it)$
- GPU pain: **thread divergence** — columns converge at different rates.

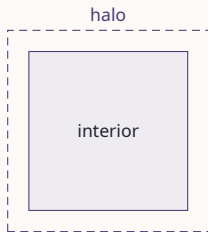
3 Motif 5 — Barotropic subcycle (2-D split-explicit)

fast surface mode, small steps

```
do m = 1, n_bt                                ! n_bt ~ 10-60, dt_bt = dt/n_bt
  eta = eta - dt_bt * div(D*U)                ! 2-D continuity (pointwise + halo)
  U = U - dt_bt * (g*grad(eta) + ...)         ! 2-D momentum (stencil)
  call halo_exchange(eta, U)                  ! every substep
end do
```

- Cheap per substep but **sequential** in m . Work $\Theta(Hn_{bt}) \cdot \text{depth } \Theta(n_{bt})$
- Tiny 2-D kernels $\times n_{bt}$ launches $\times$ halo each $\Rightarrow$ **launch-latency bound**.

3 Motif 6 — Halo exchange / communication



- Not a compute kernel — the glue. Exchange tile-edge faces with 4 (or 8) neighbours.
- $\text{comm } \Theta(\text{surface}) = \Theta(\text{halo} \cdot (n_i + n_j) \cdot n_z / \sqrt{P})$ vs compute $\Theta(N/P)$
- Strong scaling set by surface/volume; GPUDirect avoids host staging.



4

CPU & GPU parity

4

Many small launches

Kernels rarely share bounds

MOM6 kernels

```
do concurrent (bounds_0)
  call do_ops(...)
end do
do concurrent (bounds_1)
  call do_ops(...)
end do
! ... many more
```

- Each launch adds **microseconds** of gap on the GPU.
- Readability beats fusing everything into one obfuscated kernel.
- Fix: !\$acc kernels async on NVIDIA. (!\$omp nowait is flaky.)

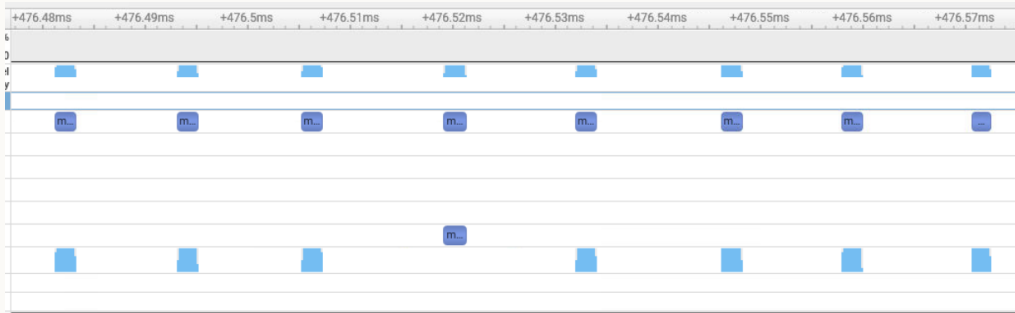
Evidence

nsys timelines on the next slide.

4

The gaps are real

nsys timeline, microsecond scale — synchronous launches

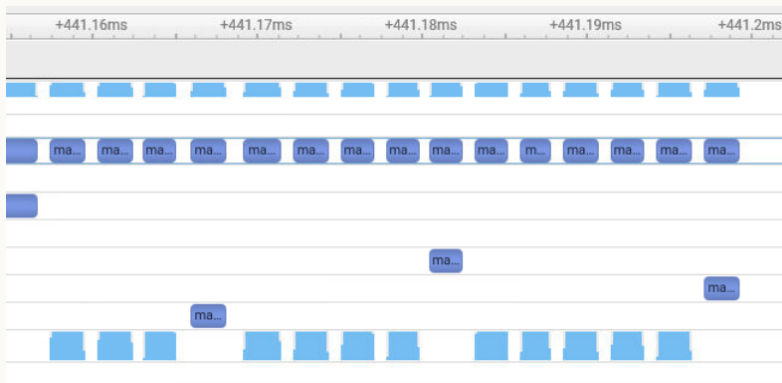


- Each kernel waits for the last — **idle gaps** between back-to-back do concurrent launches.
- At microsecond kernels, the launch latency dominates.

4

async closes them

same nsys window — !\$acc kernels async



- !\$acc kernels async lets independent kernels queue back-to-back — the gaps collapse.
- !\$omp nowait is the portable equivalent, but flaky in practice today.

4 CPU and GPU want opposite layouts

Column-based — CPU

- outer (i, j) , inner k
- great cache reuse
- limited GPU parallelism: a whole column per thread $\Rightarrow$ **~ 256 registers, 12.5% occupancy**

Plane-based — GPU

- collapse (i, j, k)
- full GPU parallelism
- cache misses degrade the CPU

How do we recover *both* from one source?

after Kommera & Appelhans, NVIDIA

hybrid strided loop — one knob

```
do j_out = 1, nj, stride           ! outer: strided, CPU sequential
  do i_out = 1, ni, stride
    do concurrent (jb=0:stride-1,ib=0:stride-1,k=1:nk)
      i = i_out+ib ; j = j_out+jb
      call compute_column(i,j,k)    ! pure
    end do
  end do
end do
```

- $\text{stride}=1 \rightarrow \text{column-based (CPU)}$ · $\text{stride}=\text{nCells} \rightarrow \text{plane-based (GPU)}$.
- In MOM6 we block the **vertical** instead: nz into bk -level blocks, one `do concurrent` each — one knob per device, no duplicated kernels.

5

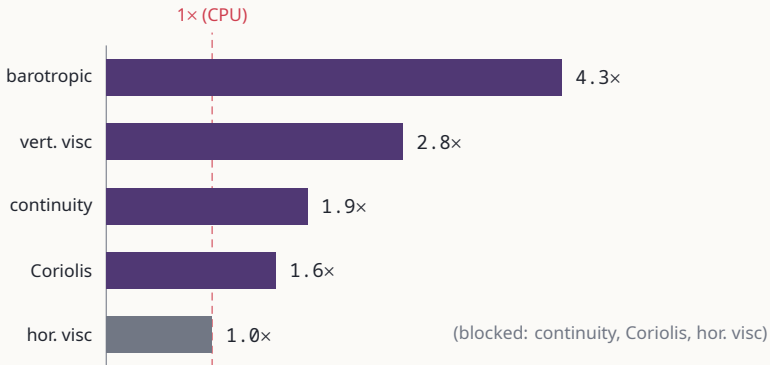
Results



5

Per-kernel GPU speedup

1 H200 vs a 52-core Sapphire Rapids node · domain $300 \times 300 \times 100$

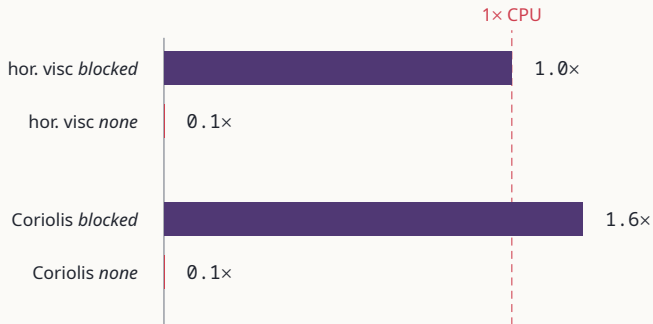


Every ported dycore kernel meets or beats the full CPU node on a **single** GPU.

5

Blocking vs no-blocking

column-bound kernels, 1 H200 vs 52-core SPR node — log scale



- Without blocking, **hor. visc** and **Coriolis** ran at only $\sim 0.1\times$ the CPU — $\approx 10\times$ slower.
- With k -blocking they recover to **1.0x** and **1.6x** — parity or better, on a single GPU.
- Same source; only the bk knob changes.

5 Multi-GPU was almost free

one rank → one GPU — the FMS/MPI decomposition already existed

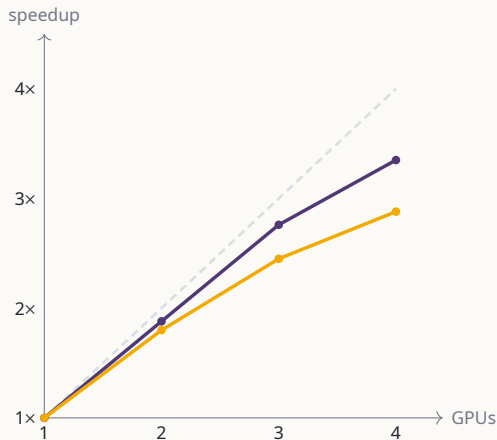
the entire multi-GPU enablement

```
call MPI_COMM_RANK( mpp_comm_private, pe, error )
call MPI_COMM_SIZE( mpp_comm_private, npes, error )
!$  call omp_set_default_device (pe)
!$acc set device_num(pe)
```

- Each MPI rank binds to its own GPU (pe → device) — that is the whole change.
- Domain decomposition and halo exchange were **already there** in FMS; nothing else to add.
- GPUDirect lets the existing pass_* halos travel device-to-device.

5 Strong scaling, multi-GPU

GPUDirect halo exchange · speedup vs 1 GPU



— V100 (800²×100)

— H200 (1248²×200)

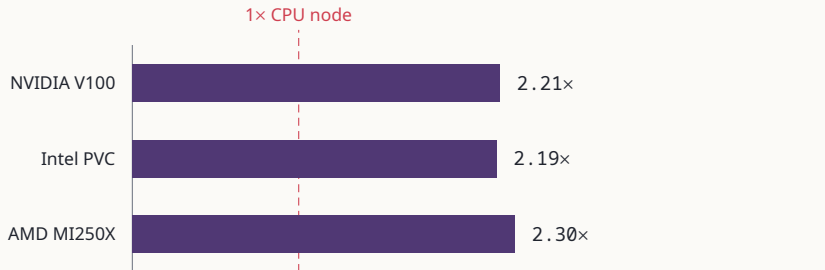
— ideal (linear)

- Up to **3.35×** on 4 GPUs.
- Parallel efficiency at 4 GPUs: **84%** (V100), **72%** (H200).

5

Cross-vendor portability

Double-gyre miniapp · 1 GPU each · vs 104-core SPR node (536.8 s) · Monash Hackathon, Mar 2026



One source code → **three vendors.** do concurrent + OpenMP data movement, comparable speedups.

5 Conclusions

- The **dynamical core** of MOM6 runs on GPUs, with good performance on *both* CPUs and GPUs — from one source.
- `do concurrent` + OpenMP mapping is **portable** across NVIDIA, Intel and AMD.
- A single knob (`bk / stride`) recovers CPU and GPU performance without duplicated kernels.
- Bitwise reproducibility preserved.

Next

Parametrizations and physics are underway — GPU-accelerated MOM6 at a cinema near you soon.

Questions?

Thank you

GPU-accelerated MOM6 dycore — physics underway.

Jorge Luis Gálvez Vallejo (NCI)

with Edward Yang (ACCESS-NRI), Utheri Wagura (SAIC) & Marshall Ward (NOAA-GFDL)

jorge.galvezvallejo@anu.edu.au



MOM6 (dev/gpu)

